

Project 8

Annotation Microservice & React Frontend

23070122005 — Arya Balwadkar

NOTE This section picks up mid-project — it covers the Annotation microservice and the React frontend only. The Auth microservice and earlier setup steps referenced here (e.g. the JWT token) are assumed to already be running from earlier in the project.

PHASE 1 The Annotation Microservice

This microservice acts as the core processing engine. It requires token verification from the Auth service to access endpoints, applies rule-based logic to auto-detect confidential patterns (such as emails and phone numbers), and manages dataset records.

1 Write the Annotation API Code

1. Navigate into the annotation-api directory.
2. Create a new file named server.js.
3. Add the following implementation:

JAVASCRIPT – server.js

```
const express = require('express');
const jwt = require('jsonwebtoken');
const cors = require('cors');

const app = express();
app.use(express.json());
app.use(cors());

const JWT_SECRET = process.env.JWT_SECRET || 'local-dev-secret-key';
const PORT = process.env.PORT || 5002;

// Middleware: Verify JWT Token
const authenticateToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'Access denied: No token provided' });
  }
}
```

```

    jwt.verify(token, JWT_SECRET, (err, user) => {
      if (err) {
        return res.status(403).json({ error: 'Forbidden: Invalid or expired token'
});
      }
      req.user = user;
      next();
    });
  });
};

// Mock In-Memory Store
let datasets = [
  {
    id: 1,
    title: 'Customer Feedback Sample',
    text: 'Please contact support at support@example.com or call +1-555-0199 for
billing issues.',
    annotations: []
  }
];

// Rule-Based Engine to auto-flag confidential patterns
const detectConfidentialPatterns = (text) => {
  const rules = [
    { type: 'EMAIL', regex: /\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,7}\b/g
},
    { type: 'PHONE', regex: /\b\+?[0-9]{1,3}?[-. ]?\(?[0-9]{3}\)?[-. ]?[0-9]{3}[-.
]?[0-9]{4}\b/g }
  ];
  let detected = [];
  rules.forEach((rule) => {
    let match;
    while ((match = rule.regex.exec(text)) !== null) {
      detected.push({
        type: rule.type,
        value: match[0],
        index: match.index
      });
    }
  });
};

```

```

    }
  });
  return detected;
};

// Route: Get Datasets (Protected)
app.get('/api/datasets', authenticateToken, (req, res) => {
  res.json(datasets);
});

// Route: Auto-Scan Text for Confidential Data (Protected)
app.post('/api/datasets/:id/scan', authenticateToken, (req, res) => {
  const dataset = datasets.find(d => d.id === parseInt(req.params.id));
  if (!dataset) return res.status(404).json({ error: 'Dataset not found' });

  const findings = detectConfidentialPatterns(dataset.text);
  dataset.annotations = findings;

  res.json({ message: 'Scan complete', findings, dataset });
});

// Health check route for Kubernetes Readiness Probes
app.get('/health', (req, res) => res.status(200).json({ status: 'Annotation API is healthy' }));

app.listen(PORT, () => {
  console.log(`🌀 Annotation Engine Microservice active on port ${PORT}`);
});

```

2 Test the Annotation Engine Locally

Start the Annotation Engine — in your PowerShell terminal inside the annotation-api directory, run:

POWERSHELL

```
node server.js
```

It should log: 🌀 Annotation Engine Microservice active on port 5002

Test scanning with the token — using your second PowerShell window, run the scan endpoint using the token obtained from the Auth Service. Replace <YOUR_JWT_TOKEN> with the token received earlier:

POWERSHELL

```
curl.exe -X POST http://localhost:5002/api/datasets/1/scan -H "Authorization: Bearer <YOUR_JWT_TOK
```

This command is shown exactly as it appears in the source document — the token value was cut off there.

PHASE 2 The React Frontend

1 Write the React Code

Navigate to the frontend folder — open a new PowerShell terminal and go into the React app folder you scaffolded earlier:

POWERSHELL

```
cd annotation-ui
```

Update the application logic — open `src/App.js` in your code editor. Delete everything in that file and paste this clean, single-page dashboard code:

JAVASCRIPT – src/App.js

```
import React, { useState } from 'react';

function App() {
  const [token, setToken] = useState('');
  const [datasets, setDatasets] = useState([]);

  // 1. Talk to the Auth Microservice
  const handleLogin = async () => {
    const res = await fetch('http://localhost:5001/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username: 'annotator', password: 'secure123' })
    });
    const data = await res.json();
    if (data.token) {
      setToken(data.token);
      fetchDatasets(data.token); // Fetch data once logged in
    } else {
      alert('Login failed. Is the Auth API running?');
    }
  };
}
```

```

// 2. Talk to the Annotation Microservice
const fetchDatasets = async (authToken) => {
  const res = await fetch('http://localhost:5002/api/datasets', {
    headers: { 'Authorization': `Bearer ${authToken}` }
  });
  const data = await res.json();
  setDatasets(data);
};

// 3. Trigger the Rule-Based Engine
const handleScan = async (id) => {
  const res = await fetch(`http://localhost:5002/api/datasets/${id}/scan`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${token}` }
  });
  const data = await res.json();
  // Update the UI with the newly detected confidential flags
  setDatasets([data.dataset]);
};

return (
  <div style={{ padding: '40px', fontFamily: 'sans-serif', maxWidth: '800px', margin:
'0 auto' }}>
    <h1>Data Annotation Engine 🕒</h1>
    {!token ? (
      <button onClick={handleLogin} style={{ padding: '10px 20px', fontSize: '16px',
cursor: 'pointer' }}>
        Secure Login (Annotator)
      </button>
    ) : (
      <div>
        <h2>Active Datasets</h2>
        {datasets.map(ds => (
          <div key={ds.id} style={{ border: '1px solid #ccc', padding: '20px',
borderRadius: '8px' }}>
            <h3 style={{ marginTop: 0 }}>{ds.title}</h3>
            <p style={{ fontSize: '18px', lineHeight: '1.5' }}>{ds.text}</p>
            <button
              onClick={() => handleScan(ds.id)}

```

```

        style={{ padding: '8px 16px', backgroundColor: '#007bff', color:
'white', border: 'none', borderRadius: '4px', cursor: 'pointer' }}>
        Auto-Scan for Confidential Data
      </button>
      {ds.annotations && ds.annotations.length > 0 && (
        <div style={{ marginTop: '20px', backgroundColor: '#fff3cd', padding:
'15px', borderRadius: '4px', borderLeft: '4px solid #ffc107' }}>
          <strong style={{ color: '#856404' }}><img alt="Warning icon" data-bbox="308 218 322 232"/> Detected Confidential
Anomalies:</strong>
          <ul style={{ margin: '10px 0 0 0' }}>
            {ds.annotations.map((ann, idx) => (
              <li key={idx}><strong>[{ann.type}]</strong> - {ann.value}</li>
            ))}
          </ul>
        </div>
      )}
    </div>
  )}
</div>
);
}

export default App;

```

2 Test the Full Flow

Before we run this, ensure your two backend terminals are still open and running (node index.js for Auth on 5001, and node server.js for Annotation on 5002).

In your annotation-ui terminal, start the React frontend:

```

POWERSHELL
npm start

```

This should automatically open your browser to <http://localhost:3000>.

